

1-Hour Practical Project: AI Support Ticket Workflow

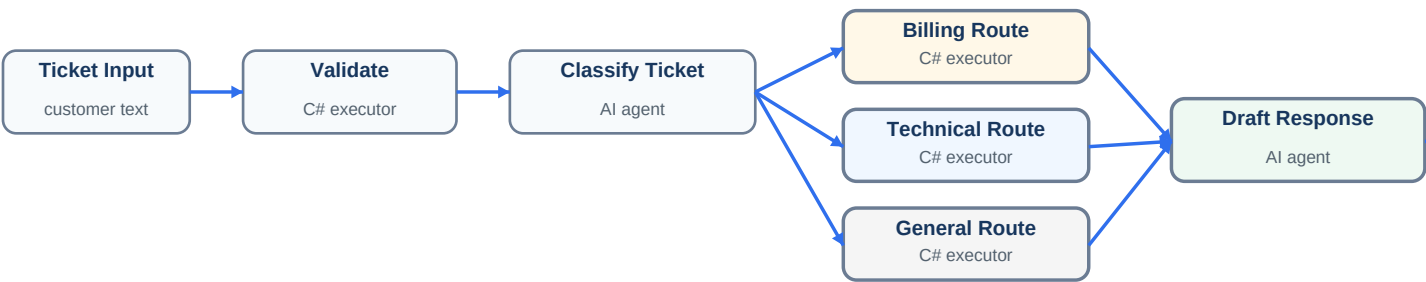
Microsoft Agent Framework Workflows | Common use case: customer-support ticket triage

Build a small console application that takes a customer support message and moves it through a deterministic workflow. Some steps use normal C# logic and one step uses an AI agent. The goal is to practice executors, edges, branching, workflow state/output, and the idea of combining deterministic business logic with AI reasoning.

Scenario

A software company receives support tickets such as: *"I was charged twice for my subscription and need a refund."* The application should classify the ticket, route it to the correct specialist path, create a short professional response, and produce a final ticket result.

Workflow Diagram



Branch according to category: Billing / Technical / General

Required Steps

Step	Type	Requirement
1. Ticket Input	Input	Read a support message from the console.
2. Validate	C# executor	Reject empty/very short tickets. Otherwise forward the ticket.
3. Classify	AI agent	Return one category: Billing, Technical, or General, plus a one-sentence summary.
4. Route	Conditional edge / branch	Send the classified ticket to the matching route.
5. Specialist Route	C# executor	Attach a team name and simple handling instruction based on the category.
6. Draft Response	AI agent	Generate a concise customer-facing reply using the classification and route information.
7. Final Result	Output	Print category, assigned team, summary, and draft reply.

Suggested Data Model

SupportTicket: Message
TicketClassification: Category, Summary
RoutedTicket: Message, Category, Summary, AssignedTeam, HandlingInstruction
TicketResult: Category, AssignedTeam, Summary, DraftResponse

Workflow Construction

Use WorkflowBuilder with the validation executor as the starting point. Connect executors with direct edges and use conditional routing after classification. Agent Framework workflows are directed graphs made of executors and edges; agents can also participate as workflow executors. Keep the routing deterministic: the AI classifies the ticket, while your workflow decides which path executes.

Acceptance Tests

Billing: "I was charged twice this month." -> Billing -> Billing Team.

Technical: "The app crashes whenever I upload a PDF." -> Technical -> Engineering Support.

General: "Do you offer discounts for universities?" -> General -> Customer Success.

Validation: empty input -> workflow should stop/reject without calling the classification agent.

1-Hour Implementation Plan

Time	Work
0-10 min	Create console project, records/classes, and two simple deterministic executors.
10-25 min	Create the classification agent and structured classification output.
25-40 min	Build WorkflowBuilder graph and conditional routing.
40-50 min	Add the response-writing agent and final output.
50-60 min	Run the four acceptance tests, clean code, and add the workflow diagram to README.

Keep It Small

Do not add a database, authentication, React/Blazor UI, RAG, MCP, email sending, or ticket-system integration. Those are good later extensions, but they would distract from the purpose of this exercise: learning workflow orchestration.

Definition of Done

The console accepts a ticket, validates it, uses an AI agent to classify it, follows exactly one category branch, uses an AI agent to draft the response, and prints a structured final result. Your README should include the workflow diagram and one sample run.

Optional 15-Minute Extension

Add a **Human Approval** step only for Billing tickets before the final response is produced. This introduces the human-in-the-loop idea without making the base project much larger.